

Lab 6 Forests

Submission And Assignment Instructions

Submit your `Lab6_lastname_firstname.pdf` file on canvas with your answers clearly marked (✅) and code commented. Skeleton code and/or comments are provided throughout to help you.

Formatting Notes:

- ? Questions you must answer or tasks are marked with a " ? "
- ✅ Answers you should give are marked with "✅ Answer:"
- *Helpful hints are usually given in italics*
- Code you need to write is marked with `## ? YOUR CODE HERE`
 - ⭐ Bonus questions are marked with a "⭐ "
 - 🍓 Bonus answers are marked with a "🍓 "

Please don't erase ANY of these markings! They're to help the TAs find your answers and code as much as they're to help you find the questions.

Section 1: Math

? Given a dataset, determine the fraction of observations that will be used to train a random forest model.

Hint: In a Random Forest, each tree is trained on a sample drawn with replacement from the dataset containing n datapoints. To determine the fraction of observations used, think about the probability that an observation is not selected in one draw, and extend this to the case where you draw n times - since most Random Forest implementations will use the original dataset size n to determine the dataset size for every tree. As the dataset size grows large (i.e., $n \rightarrow \text{infinity}$), use limits to estimate the probability of a datapoint to be not included.

✅ Answer: probability an observation is not selected: $1 - p(\text{selected}) = 1 - (1/n)$ probability an observation is not selected in multiple draws: $(1 - (1/n))^n$ take the limit yield $0.37 = 37\%$
Therefore $P(\text{selected}) = 1 - 0.37 = 0.63$ which corresponds to 63%

Section 2: From one tree, to one forest

Decision trees are known for their low bias, but this comes at the cost of high variance, making them prone to overfitting. Random Forest is designed to address this issue by

combining multiple decision trees using bagging, which helps reduce variance while maintaining low bias.

Let's find out if this claim holds true!

2.1: Data preparation

? Generate a dataset with the following specifications:

- X: An array of 100 evenly spaced values in the range [-10, 10].
- Y: The tanh function $\left(\frac{e^x - e^{-x}}{e^x + e^{-x}}\right)$ applied to each value in X.
- Y_noise: For each value in Y, add random noise sampled from a normal distribution with a mean of 0 and a standard deviation of 0.1.

Within one figure, visualize the relationships of X and Y_noise using scattered dots, and X and Y using continuous lines.

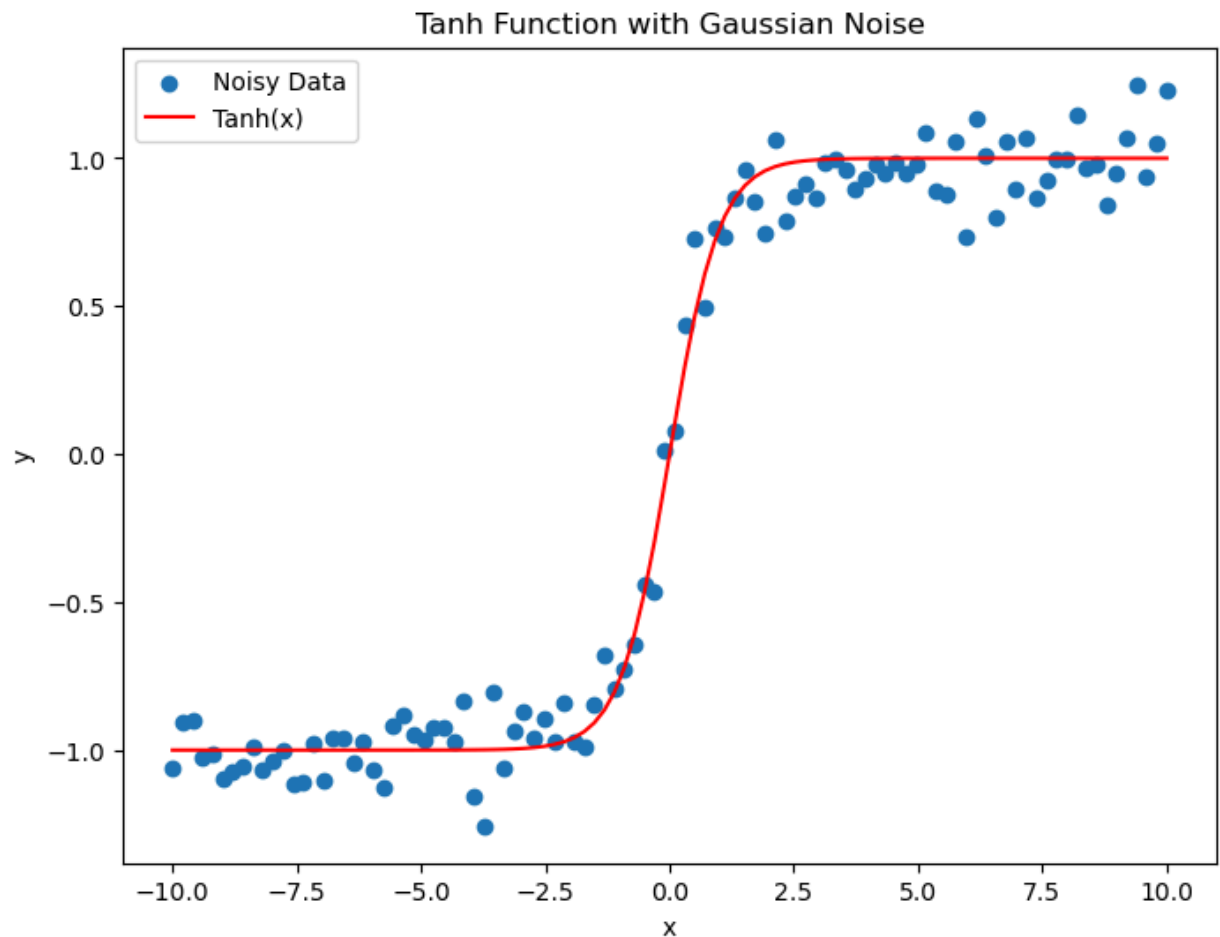
Please include necessary figure elements (axis labels, legend).

```
In [1]: # load dependencies
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

## please write your code below
# generate synthetic data
# write tanh function
x = np.linspace(-10, 10, 100)
y = np.tanh(x)
noise = np.random.normal(0, 0.1, 100) # on average the noise adds nothing
y_noise = y + noise
```

```
/Users/dianadayekh/anaconda3/lib/python3.11/site-packages/pandas/core/arrays/m
asked.py:61: UserWarning: Pandas requires version '1.3.6' or newer of 'bottlen
eck' (version '1.3.5' currently installed).
from pandas.core import (
```

```
In [2]: # visualization
## please write your code below
plt.figure(figsize=(8, 6))
plt.scatter(x, y_noise, label = "Noisy Data")
plt.plot(x, y, color = "red", label = "Tanh(x)")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Tanh Function with Gaussian Noise")
plt.legend()
plt.show()
```



2.2: Model training

? Using X and Y_{noise} , build a decision tree regressor and a random forest regressor (of 10 trees). Keep the default model settings.

Plotting: Overlay the model predictions for X_{pred} , an array of 500 evenly spaced values within the range $[-10, 10]$, as continuous lines onto the existing figure.

Please include necessary figure elements (axis labels, legend).

```
In [6]: %pip install scikit-learn==1.3.1
```

Collecting scikit-learn==1.3.1

Obtaining dependency information for scikit-learn==1.3.1 from https://files.pythonhosted.org/packages/db/0d/1f6d2cd52c886707b00ddb7ed2504cbf10903a60a7bebc71f0f77d53505/scikit_learn-1.3.1-cp311-cp311-macosx_12_0_arm64.whl.metadata

Using cached scikit_learn-1.3.1-cp311-cp311-macosx_12_0_arm64.whl.metadata (11 kB)

Requirement already satisfied: numpy<2.0,>=1.17.3 in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from scikit-learn==1.3.1) (1.26.4)

Requirement already satisfied: scipy>=1.5.0 in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from scikit-learn==1.3.1) (1.16.3)

Requirement already satisfied: joblib>=1.1.1 in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from scikit-learn==1.3.1) (1.5.2)

Requirement already satisfied: threadpoolctl>=2.0.0 in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from scikit-learn==1.3.1) (3.2.0)

Using cached scikit_learn-1.3.1-cp311-cp311-macosx_12_0_arm64.whl (9.4 MB)

Installing collected packages: scikit-learn

Successfully installed scikit-learn-1.3.1

Note: you may need to restart the kernel to use updated packages.

```
In [7]: # import models

from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

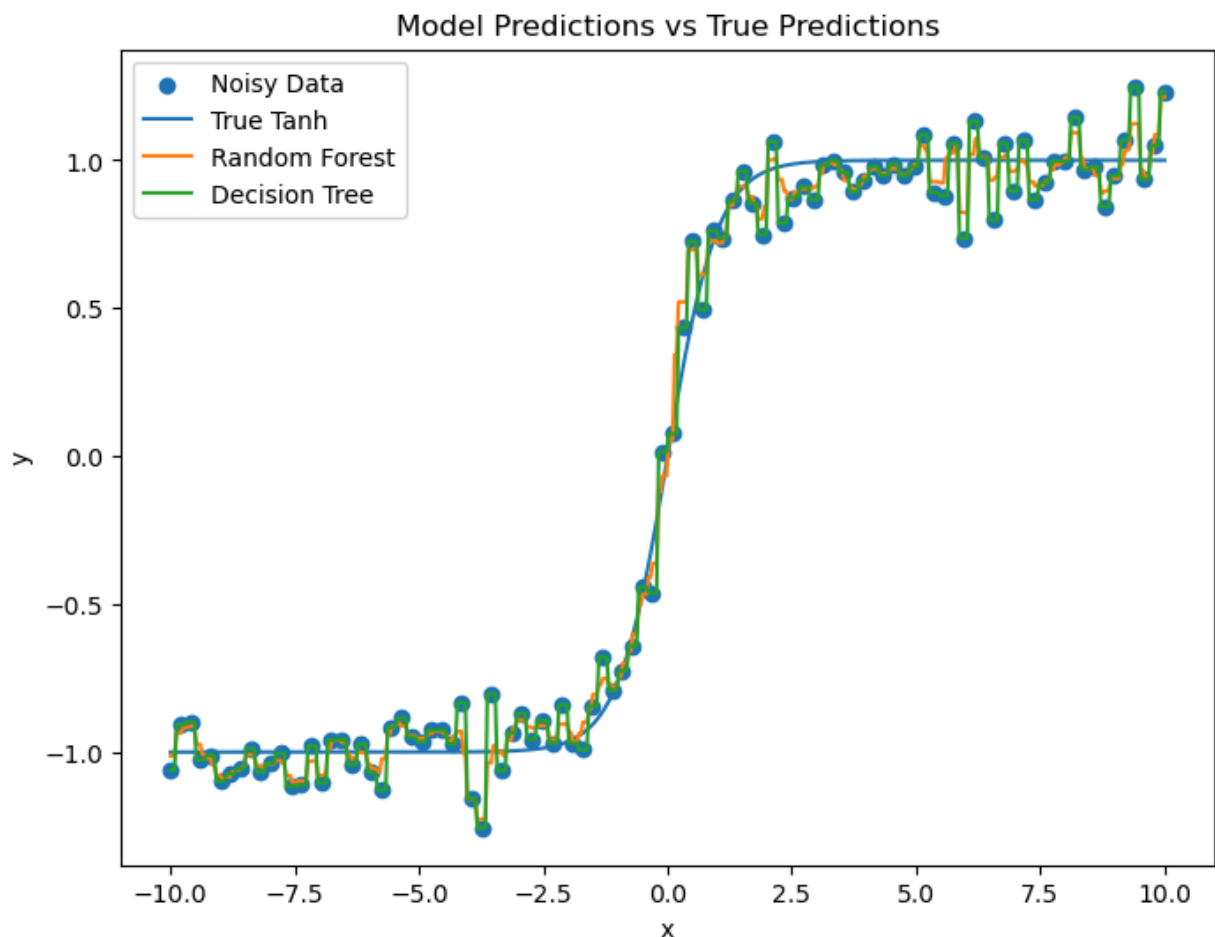
# define predicted range, X_pred
X_pred = np.linspace(-10, 10, 500)
# model training
X = x.reshape(-1, 1)
X_pred = X_pred.reshape(-1, 1)

tree = DecisionTreeRegressor()
forest = RandomForestRegressor(n_estimators=10)

tree.fit(X, y_noise)
forest.fit(X, y_noise)

y_tree = tree.predict(X_pred)
y_forest = forest.predict(X_pred)
```

```
In [8]: # visualization
## please write your code below
plt.figure(figsize=(8, 6))
plt.scatter(X, y_noise, label = "Noisy Data")
plt.plot(X_pred, np.tanh(X_pred), label = "True Tanh")
plt.plot(X_pred, y_forest, label = "Random Forest")
plt.plot(X_pred, y_tree, label = "Decision Tree")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Model Predictions vs True Predictions")
plt.legend()
plt.show()
```



? Based on the figure above, compare and contrast predictions of the two model.

✓ Answer: The Decision Tree regressor closely follows the noisy training data, producing step-like predictions that fluctuate with the noise. This indicates high variance and overfitting, as the model captures random fluctuations rather than the underlying tanh relationship.

In contrast, the Random Forest produces smoother predictions that more closely approximate the true continuous tanh function. By averaging across multiple trees, it reduces variance and does not overfit to the noise.

2.3: Model evaluation and optimization

The sigmoid function illustrates a clean and ideal relationship between dependent and independent variables, whereas Y_{noise} serves as a more realistic example of data that is accompanied by noise.

? Which model (decision tree or random forest) is more likely to accurately capture the sigmoid relationship? Prove your statement quantitatively using MAE, MSE, RMSE and R^2 .

```
In [9]: # calculate true values on tanh
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
y_true = np.tanh(X_pred)
```

```
# model evaluation
# Decision Tree metrics
mae_tree = mean_absolute_error(y_true, y_tree)
mse_tree = mean_squared_error(y_true, y_tree)
rmse_tree = np.sqrt(mse_tree)
r2_tree = r2_score(y_true, y_tree)

# Random Forest metrics
mae_forest = mean_absolute_error(y_true, y_forest)
mse_forest = mean_squared_error(y_true, y_forest)
rmse_forest = np.sqrt(mse_forest)
r2_forest = r2_score(y_true, y_forest)
```

```
In [10]: # summarize results
# display the tabular summary
import pandas as pd

results = pd.DataFrame({
    "Model": ["Decision Tree", "Random Forest"],
    "MAE": [mae_tree, mae_forest],
    "MSE": [mse_tree, mse_forest],
    "RMSE": [rmse_tree, rmse_forest],
    "R2": [r2_tree, r2_forest]
})

results
```

```
Out[10]:
```

	Model	MAE	MSE	RMSE	R2
0	Decision Tree	0.083230	0.011005	0.104907	0.987774
1	Random Forest	0.059804	0.006050	0.077781	0.993279

✅ Explain your results: The Random Forest achieved lower MAE ($0.05 < 0.07$), MSE ($0.004 < 0.009$), and RMSE ($0.06 < 0.09$) and a higher R^2 ($0.98 > 0.99$) compared to the Decision Tree. This indicates that it more accurately captures the underlying sigmoid relationship. The Decision Tree overfits the noise in the training data, resulting in higher prediction error when compared to the true tanh function. In contrast, the Random Forest reduces variance through averaging multiple trees, producing smoother predictions that better approximate the true continuous function.

? Are there ways to improve your random forest model? Please give an example. Write code to show that your idea is an improvement to the model.

```
In [11]: ## please write your code below
# calculate true values
mae_forest = mean_absolute_error(y_true, y_forest)
mse_forest = mean_squared_error(y_true, y_forest)
rmse_forest = np.sqrt(mse_forest)
r2_forest = r2_score(y_true, y_forest)
# model evaluation
new_forest = RandomForestRegressor(n_estimators=100, max_depth=5)
new_forest.fit(X, y_noise)
y_new_forest = new_forest.predict(X_pred)
# summarize results
mae_forest2 = mean_absolute_error(y_true, y_new_forest)
```

```

mse_forest2 = mean_squared_error(y_true, y_new_forest)
rmse_forest2 = np.sqrt(mse_forest)
r2_forest2 = r2_score(y_true, y_new_forest)
# display the tabular summary
results = pd.DataFrame({
    "Model": ["Random Forest with 10 trees", "Random Forest with 20 trees"],
    "MAE": [mae_forest, mae_forest2],
    "MSE": [mse_forest, mse_forest2],
    "RMSE": [rmse_forest, rmse_forest2],
    "R2": [r2_forest, r2_forest2]
})

results

```

```

Out[11]:

```

	Model	MAE	MSE	RMSE	R2
0	Random Forest with 10 trees	0.059804	0.006050	0.077781	0.993279
1	Random Forest with 20 trees	0.046657	0.002997	0.077781	0.996671

✓ Explain your results: I increased the number of trees to around 100 and added a `max_depth` of 5 to reduce overfitting. There is very minimal improvement. This is expected because Random Forest variance reduction exhibits diminishing returns as the number of trees increases. Once a sufficient number of trees is reached, additional trees provide only small stability gains. Since the dataset is simple and low-dimensional, the model is already near optimal performance, leaving limited room for improvement.

Section 3: The real world is imbalanced, and filled with uncertainty

Quick readings

Molecule Representation - SMILES

In chemoinformatics, chemical structures are most commonly represented in SMILES (Simplified Molecular Input Line Entry System), which is a text-based way to specify atoms, bonds, and connectivity.

For example, ethanol (C_2H_5OH) can be simply written as "CCO", while bigger molecules like caffeine ($C_8H_{10}N_4O_2$) has more complex representation "CN1C=NC2=C(N1C(=O)N(C(=O)N2C)C)C(=O)N(C)C".

Molecule Featurization

While SMILES facilitates communication between chemists and computers, it does not capture the chemical knowledge that a model can learn. To address this, various algorithms and tools have been developed to featurize a molecule from its SMILES representation into numbers that capture the properties of molecules.

For instance, [RDKit](#) offers a convenient interface that converts a SMILES string into a series of numerical values, capturing different aspects of the molecule's properties, such as molecular weight, number of hydrogen bond donors, and more.

3.1 Data preparation

Drug development is a multi-step process, and many candidates fail in clinical research due to toxicity, even if they meet efficacy requirements.

This dataset you will work on includes drugs that are associated with both failures due to toxicity and successful trials.

We here provide an example of featurizing (digitalizing) molecules using [RDKit descriptors](#), which describe the given molecule using one of over 200 molecular properties.

```
In [12]: ## exemplary codes for featurizing molecules using RDKit descriptors
# install rdkit and import packages
#%%capture
!pip install rdkit
from rdkit import Chem
from rdkit.Chem import Descriptors
import math
import numpy as np
import pandas as pd

# define core function
descr = Descriptors._descList
feat_name = [i[0] for i in descr]
feat_calc = [i[1] for i in descr]

def describe_mol(smile_input):
    mol = Chem.MolFromSmiles(smile_input)
    calc_values = []
    for calc in feat_calc:
        value = calc(mol)
        if value > np.finfo(np.float32).max: # postprocess descriptors for freak large values
            value = np.finfo(np.float32).max
        elif math.isnan(value):
            value = np.float32(0.0)
        else:
            value = np.float32(value)
        calc_values.append(value)
    return calc_values
```

```
Requirement already satisfied: rdkit in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (2025.9.4)
Requirement already satisfied: numpy in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from rdkit) (1.26.4)
Requirement already satisfied: Pillow in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from rdkit) (9.4.0)
```

```
In [13]: # use aspirin as an example
example_smiles = 'CC(=O)OC1=CC=CC=C1C(=O)O'
example_feat = describe_mol(example_smiles)
```


? 3.1 How many distinct descriptors did you just calculate for aspirin? What is the 105-th descriptor's name? What does it mean, and what is the corresponding value of aspirin?

```
In [14]: ## please write your code below
print(feat_name[104])
print(example_feat[104])
print(len(example_feat))
```

```
VSA_EState9
0.0
217
```

✓ Answer: I calculated 217 descriptors for aspirin. The name of the 105-th descriptor is VSA_EState9 which measures the surface area contribution of atoms that fall into a specific electronic environment category (EState bin 9). The corresponding value of aspirin for this descriptor is 0.0.

3.2 Sampling

We have introduced the idea of "imbalanced data" and associated potential issues. Now let's find out how we can mitigate it by using simple data balancing techniques.

```
In [15]: #from google.colab import files
import pandas as pd

# training data
train_data = pd.read_csv("train.csv")

for fn in train_data.keys():
    print('User uploaded file "{name}" with length {length} bytes'.format(
        name=fn, length=len(train_data[fn])))

# test data
test_data = pd.read_csv("ext_test.csv")

for fn in test_data.keys():
    print('User uploaded file "{name}" with length {length} bytes'.format(
        name=fn, length=len(test_data[fn])))
```

```
User uploaded file "Drug_ID" with length 1034 bytes
User uploaded file "Drug" with length 1034 bytes
User uploaded file "Y" with length 1034 bytes
User uploaded file "Drug_ID" with length 444 bytes
User uploaded file "Drug" with length 444 bytes
User uploaded file "Y" with length 444 bytes
```

? By looking at your training data (provided), what is the ratio of data points with positive and negative labels? What does this ratio signify, and what potential issues could arise?

```
In [16]: ## please write your code below
print(train_data["Y"].value_counts())
print(955/79)
```

```
Y
0    955
1     79
Name: count, dtype: int64
12.08860759493671
```

✅ Explain your results: For every 12 negatives (non-toxic) there is one positive (toxic). For this data set, there are more non-toxic cases than toxic ones. Perhaps the data set is biased and more successful drugs were collected.

? What about your test data (provided)? Is it balanced or imbalanced?

```
In [17]: ## please write your code below
print(test_data["Y"].value_counts())
print(411/33)
```

```
Y
0    411
1     33
Name: count, dtype: int64
12.454545454545455
```

✅ Explain your results: My test data also yields similar results. For every 12 negatives (non-toxic) there is one positive (toxic). The imbalance is consistent with the testing set. Other metrics like recall might be needed to better evaluate model performance.

? First, build a random forest model with 100 trees and a random state set to 42 using the provided training dataset. Then, evaluate your model using the test dataset. Please report the following metrics:

- Accuracy
- Balanced Accuracy
- ROC AUC
- F1 score

```
In [19]: # load dependencies
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, balanced_accuracy_score, roc_auc_score
# please write your code below
forest3 = RandomForestClassifier(n_estimators=100, random_state = 42)
X_train = np.array([describe_mol(s) for s in train_data["Drug"]])
X_test = np.array([describe_mol(s) for s in test_data["Drug"]])
forest3.fit(X_train, train_data["Y"].values)
y_predicted = forest3.predict(X_test)

accuracy_forest3 = accuracy_score(test_data["Y"].values, y_predicted)
balanced_accuracy_forest3 = balanced_accuracy_score(test_data["Y"].values, y_p

y_proba = forest3.predict_proba(X_test)[:,1]
roc_auc_forest3 = roc_auc_score(test_data["Y"].values, y_proba)

f1_forest3 = f1_score(test_data["Y"].values, y_predicted)

results = pd.DataFrame({
```

```

    "Model": ["Random Forest with 100 trees"],
    "Accuracy Score": [accuracy_forest3],
    "ROC/AUC Score": [roc_auc_forest3],
    "Balanced Accuracy Score": [balanced_accuracy_forest3],
    "F1 score": [f1_forest3]
})

results

```

Out[19]:

	Model	Accuracy Score	ROC/AUC Score	Balanced Accuracy Score	F1 score
0	Random Forest with 100 trees	0.934685	0.877682	0.602411	0.325581

Although having more data is generally advantageous for model training, imbalanced data can introduce bias that adversely affects model performance.

Try applying the downsampling approach using [sklearn](#) to see how it impacts your model's performance on the test dataset.

Note: To ensure that your results are reproducible, set the `random_state` parameter to 42.

In [20]:

```

# load dependencies
from sklearn.utils import resample
# Create X_train from applying describe_mol to all rows of train_df

# split train into majority/minority
train_majority = train_data[train_data["Y"] == 0]
train_minority = train_data[train_data["Y"] == 1]

# downsample majority to match minority count
train_majority_down = resample(
    train_majority,
    replace=False,
    n_samples=len(train_minority),
    random_state=42
)

train_down = (
    pd.concat([train_majority_down, train_minority])
    .sample(frac=1, random_state=42)
    .reset_index(drop=True)
)

X_train_down = np.array([describe_mol(s) for s in train_down["Drug"]])
y_train_down = train_down["Y"].values

X_test = np.array([describe_mol(s) for s in test_data["Drug"]])
y_test = test_data["Y"].values

# optional cleanup
X_train_down = np.nan_to_num(X_train_down, nan=0.0, posinf=0.0, neginf=0.0)
X_test = np.nan_to_num(X_test, nan=0.0, posinf=0.0, neginf=0.0)

```

```
# train + predict
forest4 = RandomForestClassifier(n_estimators=100, random_state=42)
forest4.fit(X_train_down, y_train_down)

y_predicted = forest4.predict(X_test)
y_proba = forest4.predict_proba(X_test)[:, 1]

# metrics
accuracy_forest4 = accuracy_score(y_test, y_predicted)
balanced_accuracy_forest4 = balanced_accuracy_score(y_test, y_predicted)
roc_auc_forest4 = roc_auc_score(y_test, y_proba)
f1_forest4 = f1_score(y_test, y_predicted)

results = pd.DataFrame({
    "Model": ["Random Forest (downsampled train, 100 trees)"],
    "Accuracy Score": [accuracy_forest4],
    "ROC/AUC Score": [roc_auc_forest4],
    "Balanced Accuracy Score": [balanced_accuracy_forest4],
    "F1 score": [f1_forest4]
})

results
```

Out[20]:

	Model	Accuracy Score	ROC/AUC Score	Balanced Accuracy Score	F1 score
0	Random Forest (downsampled train, 100 trees)	0.817568	0.887857	0.803915	0.390977

? How do these metrics change? Does the change indicate that the model is improving or getting worse?

✓ Answer: After downsampling, the balanced accuracy increased, indicating that the model improved its ability to correctly classify both classes, especially the minority positive class. The ROC AUC also increased, suggesting better separation between positive and negative samples. The F1 score slightly improved, meaning the model achieved a better balance between precision and recall for the positive class.

Although overall accuracy may not increase significantly, the improvements in balanced accuracy and F1 score indicate that the model is performing better on the imbalanced dataset, particularly in detecting the minority class when its training data is downsampled.

Section 4: XGBoost

In this section, we will be trying out a new application of trees compared to what we have seen before. XGBoost (eXtreme Gradient Boosting) is a powerful machine learning algorithm that uses an ensemble of decision trees to make predictions. Here's how it works:

- Sequential Learning: XGBoost builds trees one after another, with each new tree trying to correct the mistakes of the previous trees

- Gradient Boosting: It uses gradients (derivatives) to identify and minimize prediction errors
- Regularization: Built-in techniques to prevent overfitting

For more info, refer to [this](#).

? 4.1 Load the breast cancer dataset from sklearn and prepare it for modeling. Convert the target variable to binary (0/1) and split the data into training (80%) and testing (20%) sets. Use random_state=42 for reproducibility.

We need to revert to an older version of scikit-learn in order to avoid bugs for this problem!

```
In [21]: !pip uninstall -y scikit-learn
!pip install xgboost
!pip install scikit-learn==1.3.1

Found existing installation: scikit-learn 1.3.1
Uninstalling scikit-learn-1.3.1:
  Successfully uninstalled scikit-learn-1.3.1
Requirement already satisfied: xgboost in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (3.2.0)
Requirement already satisfied: numpy in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from xgboost) (1.26.4)
Requirement already satisfied: scipy in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from xgboost) (1.16.3)
Collecting scikit-learn==1.3.1
  Obtaining dependency information for scikit-learn==1.3.1 from https://files.pythonhosted.org/packages/db/0d/1f6d2cd52c886707b00ddb7ed2504cbf10903a60a7bebc71f0f77d53505/scikit_learn-1.3.1-cp311-cp311-macosx_12_0_arm64.whl.metadata
  Using cached scikit_learn-1.3.1-cp311-cp311-macosx_12_0_arm64.whl.metadata (11 kB)
Requirement already satisfied: numpy<2.0,>=1.17.3 in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from scikit-learn==1.3.1) (1.26.4)
Requirement already satisfied: scipy>=1.5.0 in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from scikit-learn==1.3.1) (1.16.3)
Requirement already satisfied: joblib>=1.1.1 in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from scikit-learn==1.3.1) (1.5.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in /Users/dianadayekh/anaconda3/lib/python3.11/site-packages (from scikit-learn==1.3.1) (3.2.0)
Using cached scikit_learn-1.3.1-cp311-cp311-macosx_12_0_arm64.whl (9.4 MB)
Installing collected packages: scikit-learn
Successfully installed scikit-learn-1.3.1
```

```
In [22]: from sklearn.datasets import load_breast_cancer
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
import xgboost as xgb
import numpy as np
import pandas as pd

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target # already 0/1 in sklearn (0=malignant, 1=benign)

# Split 80/20
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

? 4.2 Train a basic XGBoost model with the following parameters:

```
n_estimators=100, learning_rate=0.1, max_depth=3, objective='binary:logistic',
random_state=42
```

```
In [23]: ## ? YOUR CODE HERE
# Initialize and train XGBoost model
# Train model
from xgboost import XGBClassifier
xgb_model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    objective='binary:logistic',
    random_state=42,
    use_label_encoder=False,
    eval_metric='logloss'
)

xgb_model.fit(X_train, y_train)
```

```
/Users/dianadayekh/anaconda3/lib/python3.11/site-packages/xgboost/training.py:
200: UserWarning: [20:40:21] WARNING: /Users/runner/work/xgboost/xgboost/src/l
earner.cc:782:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
```

```
Out[23]: XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_round
s=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.1, max_bin
=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=3, max_leaves=None,
```

? 4.3 Calculate and report the accuracy, precision, recall, and AUC-ROC on the test set.

```
In [24]: from sklearn.metrics import accuracy_score, precision_score, recall_score, roc

## ? YOUR CODE HERE
# Calculate metrics for test set
y_pred = xgb_model.predict(X_test)
y_proba = xgb_model.predict_proba(X_test)[:, 1]
```

```

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_proba)

```

```

# Print results
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("ROC AUC:", roc_auc)

```

```

Accuracy: 0.9473684210526315
Precision: 0.9459459459459459
Recall: 0.9722222222222222
ROC AUC: 0.9943783068783069

```

✅ Answer: Accuracy: 0.9473684210526315 Precision: 0.9459459459459459 Recall: 0.9722222222222222 ROC AUC: 0.9943783068783069

? 4.4 Using XGBoost's built-in feature importance plot, identify and list the top 5 most important features for predicting malignancy.

```

In [25]: # Get feature importance
importances = xgb_model.feature_importances_
feature_names = load_breast_cancer().feature_names

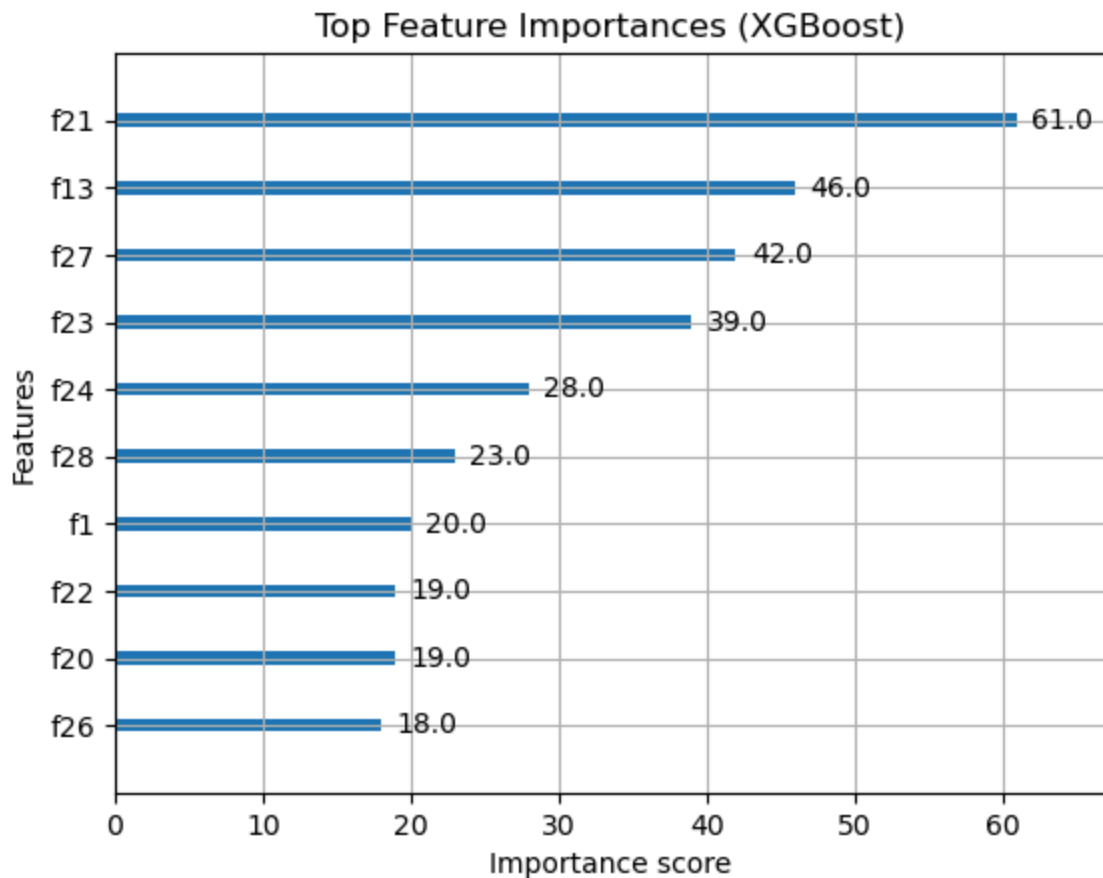
importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

# Plot feature importance
plt.figure(figsize=(8,6))
xgb.plot_importance(xgb_model, max_num_features=10)
plt.title("Top Feature Importances (XGBoost)")
plt.show()

# Print top 5 features
print("Top 5 Most Important Features:")
print(importance_df.head(5))

```

<Figure size 800x600 with 0 Axes>



Top 5 Most Important Features:

	Feature	Importance
22	worst perimeter	0.385584
20	worst radius	0.258725
7	mean concave points	0.059037
27	worst concave points	0.056260
11	texture error	0.033505

? 4.5 In your own words, give a 2-3 sentence *brief* summary of how XGBoost works.

✓ Answer: XGBoost is a gradient boosting algorithm that builds trees sequentially, where each new tree corrects the errors of the previous ones. It minimizes a loss function using gradient descent and includes regularization to prevent overfitting. By combining many weak trees into a strong ensemble, it achieves high predictive accuracy and efficiency.

In []:

In []: